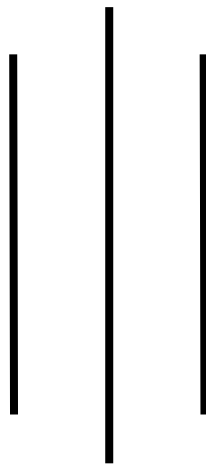


TRIBHUVAN UNIVERSITY

PATAN MULTIPLE CAMPUS

PATANDHOKA, LALITPUR



DATABASE ADMINISTRATION (BIT 352)

SUBMITTED BY:

NAME: SURESH DAHAL

ROLL: 23

CLASS: BIT-III/II

DATE: 2083/04/22

SUBMITTED TO:

BIPIN TIMALSINA

CHECKED BY

11. Working With RMAN Backup and Recovery

THEORY:

Recovery Manager (RMAN) is Oracle's built-in utility used to perform database backup, restore, and recovery operations. RMAN provides block-level corruption detection, backup optimization, and automatic recovery using archived redo logs.

For online backup and complete recovery, the database should be in **ARCHIVELOG** mode. RMAN can back up the entire database, archived redo logs, control files, and SPFILE. In case of data loss, RMAN restores the required files from backup and applies archived redo logs to recover the database to the latest consistent state.

STEPS AND COMMANDS

a) Preparation and Configuration for Backup

Step 1: Connect as SYSDBA

```
CONNECT / AS SYSDBA;
```

Step 2: Check whether the database is in ARCHIVELOG mode

```
ARCHIVE LOG LIST;
```

or

```
SELECT LOG_MODE FROM V$DATABASE;
```

Step 3: Enable ARCHIVELOG mode (if required)

```
SHUTDOWN IMMEDIATE;
```

```
STARTUP MOUNT;
```

```
ALTER DATABASE ARCHIVELOG;
```

```
ALTER DATABASE OPEN;
```

Step 4: Check Fast Recovery Area (FRA)

```
SHOW PARAMETER db_recovery_file_dest;
```

```
SHOW PARAMETER db_recovery_file_dest_size;
```

If not configured,

```
ALTER SYSTEM SET db_recovery_file_dest='D:\Oracle\FRA' SCOPE=BOTH;
```

```
ALTER SYSTEM SET db_recovery_file_dest_size=10G SCOPE=BOTH;
```

Step 5: Connect to RMAN

```
rman target /
```

Step 6: Display current RMAN configuration

```
SHOW ALL;
```

Step 7: Enable Control File Autobackup

```
CONFIGURE CONTROLFILE AUTOBACKUP ON;
```

Verify:

```
SHOW ALL;
```

b) Perform Full Backup

Step 1: Backup the entire database including archived redo logs

```
BACKUP DATABASE PLUS ARCHIVELOG;
```

Step 2: Verify backup

```
LIST BACKUP;
```

c) Simulate Database Failure

Step 1: Find the USERS datafile

```
SELECT FILE#, NAME
```

```
FROM V$DATAFILE;
```

Step 2: Shutdown database

```
SHUTDOWN IMMEDIATE;
```

Step 3: Delete the USERS datafile

Windows

```
del D:\Oracle\oradata\ORCL\users01.dbf
```

Linux

```
rm users01.dbf
```

Step 4: Switch to OCRLPDB and open database

```
alter session set container = orclpdb
```

```
alter database open
```

Oracle should display an error similar to:

```
ORA-01157
```

```
ORA-01110
```

d) Restore and Recover the Datafile

Step 1: Connect to RMAN

```
rman target /
```

Step 2: Restore the datafile

```
RESTORE DATAFILE 14;
```

Step 3: Recover the datafile

```
RECOVER DATAFILE 14;
```

Step 5: Verify

```
SELECT FILE#, STATUS
```

```
FROM V$DATAFILE
```

WHERE FILE#=14;

e) Full Database Recovery Simulation

Step 1: Shutdown database

SHUTDOWN IMMEDIATE;

Step 2: Delete the SPFILE, control files and datafiles

(Delete them using your operating system.)

Step 3: Connect RMAN

rman target /

Step 4: Restore SPFILE

STARTUP FORCE NOMOUNT;

RESTORE SPFILE FROM AUTOBACKUP;

Step 5: Restore control file

RESTORE CONTROLFILE FROM AUTOBACKUP;

Step 6: Restore database

RESTORE DATABASE;

Step 7: Recover database

RECOVER DATABASE;

Step 8: Open database

ALTER DATABASE OPEN RESETLOGS;

f) Validation and Cleanup

Step 1: Validate database

```
VALIDATE DATABASE;
```

Step 2: Check corruption

```
SELECT *  
FROM V$DATABASE_BLOCK_CORRUPTION;
```

Step 3: Verify data

```
SELECT username FROM dba_users;
```

OUTPUT

a) Preparation and Configuration for Backup

Step 1: Connect as SYSDBA

```
C:\Users\VICTUS>sqlplus / as sysdba  
  
SQL*Plus: Release 19.0.0.0.0 - Production on Sat Aug 1 07:54:38 2026  
Version 19.3.0.0.0  
  
Copyright (c) 1982, 2019, Oracle. All rights reserved.
```

Step 2: Check whether the database is in ARCHIVELOG mode

```
Name: Suresh Dahal \ Roll: 23  
PL/SQL procedure successfully completed.  
  
SQL> ARCHIVE LOG LIST;  
Database log mode                No Archive Mode  
Automatic archival               Disabled  
Archive destination              C:\Oracle_19C\RDBMS  
Oldest online log sequence       32  
Current log sequence             34
```

or

Step 3: Enable ARCHIVELOG mode

```

SQL> SHUTDOWN IMMEDIATE;
Database closed.
Database dismounted.
ORACLE instance shut down.
SQL>
SQL> STARTUP MOUNT;
ORACLE instance started.

Total System Global Area 4915720712 bytes
Fixed Size                  9277960 bytes
Variable Size              989855744 bytes
Database Buffers          3909091328 bytes
Redo Buffers                7495680 bytes
Database mounted.
SQL>
SQL> ALTER DATABASE ARCHIVELOG;

Database altered.

SQL>
SQL> ALTER DATABASE OPEN;

Database altered.

```

Step 4: Check Fast Recovery Area (FRA)

```

SQL> exec my_info;
Name: Suresh Dahal/Roll: 23/Section: A

PL/SQL procedure successfully completed.

SQL> SHOW PARAMETER db_recovery_file_dest;

```

NAME	TYPE	VALUE
db_recovery_file_dest	string	
db_recovery_file_dest_size	big integer	0

Step 5: Connect to RMAN

```

C:\Users\VICTUS>rman target /

Recovery Manager: Release 19.0.0.0.0 - Production on Sat Aug 1 08:04:16 2026
Version 19.3.0.0.0

Copyright (c) 1982, 2019, Oracle and/or its affiliates. All rights reserved.

connected to target database: ORCL (DBID=1764205357)

```

Step 6: Display current RMAN configuration

```

RMAN> show all;

using target database control file instead of recovery catalog
RMAN configuration parameters for database with db_unique_name ORCL are:
CONFIGURE RETENTION POLICY TO REDUNDANCY 1; # default
CONFIGURE BACKUP OPTIMIZATION OFF; # default
CONFIGURE DEFAULT DEVICE TYPE TO DISK; # default
CONFIGURE CONTROLFILE AUTOBACKUP ON; # default
CONFIGURE CONTROLFILE AUTOBACKUP FORMAT FOR DEVICE TYPE DISK TO '%F'; # default
CONFIGURE DEVICE TYPE DISK PARALLELISM 1 BACKUP TYPE TO BACKUPSET; # default
CONFIGURE DATAFILE BACKUP COPIES FOR DEVICE TYPE DISK TO 1; # default
CONFIGURE ARCHIVELOG BACKUP COPIES FOR DEVICE TYPE DISK TO 1; # default
CONFIGURE MAXSETSIZE TO UNLIMITED; # default
CONFIGURE ENCRYPTION FOR DATABASE OFF; # default
CONFIGURE ENCRYPTION ALGORITHM 'AES128'; # default
CONFIGURE COMPRESSION ALGORITHM 'BASIC' AS OF RELEASE 'DEFAULT' OPTIMIZE FOR LOAD TRUE ; # default
CONFIGURE RMAN OUTPUT TO KEEP FOR 7 DAYS; # default
CONFIGURE ARCHIVELOG DELETION POLICY TO NONE; # default
CONFIGURE SNAPSHOT CONTROLFILE NAME TO 'C:\ORACLE_19C\DATABASE\SNCFORCL.ORA'; # default

```

Step 7: Enable Control File Autobackup

```

RMAN> CONFIGURE CONTROLFILE AUTOBACKUP ON;

new RMAN configuration parameters:
CONFIGURE CONTROLFILE AUTOBACKUP ON;
new RMAN configuration parameters are successfully stored

```

```

RMAN> show all;

RMAN configuration parameters for database with db_unique_name ORCL are:
CONFIGURE RETENTION POLICY TO REDUNDANCY 1; # default
CONFIGURE BACKUP OPTIMIZATION OFF; # default
CONFIGURE DEFAULT DEVICE TYPE TO DISK; # default
CONFIGURE CONTROLFILE AUTOBACKUP ON;
CONFIGURE CONTROLFILE AUTOBACKUP FORMAT FOR DEVICE TYPE DISK TO '%F'; # default
CONFIGURE DEVICE TYPE DISK PARALLELISM 1 BACKUP TYPE TO BACKUPSET; # default
CONFIGURE DATAFILE BACKUP COPIES FOR DEVICE TYPE DISK TO 1; # default
CONFIGURE ARCHIVELOG BACKUP COPIES FOR DEVICE TYPE DISK TO 1; # default
CONFIGURE MAXSETSIZE TO UNLIMITED; # default
CONFIGURE ENCRYPTION FOR DATABASE OFF; # default
CONFIGURE ENCRYPTION ALGORITHM 'AES128'; # default
CONFIGURE COMPRESSION ALGORITHM 'BASIC' AS OF RELEASE 'DEFAULT' OPTIMIZE FOR LOAD TRUE ; # default
CONFIGURE RMAN OUTPUT TO KEEP FOR 7 DAYS; # default
CONFIGURE ARCHIVELOG DELETION POLICY TO NONE; # default
CONFIGURE SNAPSHOT CONTROLFILE NAME TO 'C:\ORACLE_19C\DATABASE\SNCFORCL.ORA'; # default

```

b) Perform Full Backup

Step 1: Backup the entire database including archived redo logs

```

RMAN> BACKUP DATABASE PLUS ARCHIVELOG;

Starting backup at 01-AUG-26
current log archived
allocated channel: ORA_DISK_1
channel ORA_DISK_1: SID=1351 device type=DISK
channel ORA_DISK_1: starting archived log backup set
channel ORA_DISK_1: specifying archived log(s) in backup set
input archived log thread=1 sequence=34 RECID=1 STAMP=1240128452
channel ORA_DISK_1: starting piece 1 at 01-AUG-26
channel ORA_DISK_1: finished piece 1 at 01-AUG-26
piece handle=C:\ORACLE_19C\DATABASE\014ULMU5_1_1 tag=TAG20260801T080733 comment=NONE
channel ORA_DISK_1: backup set complete, elapsed time: 00:00:01
Finished backup at 01-AUG-26

```

Step 2: Verify backup

```
RMAN> list backup;
```

List of Backup Sets
=====

BS Key	Size	Device Type	Elapsed Time	Completion Time
1	118.56M	DISK	00:00:00	01-AUG-26
BP Key: 1 Status: AVAILABLE Compressed: NO Tag: TAG20260801T080733				
Piece Name: C:\ORACLE_19C\DATABASE\014ULMU5_1_1				

List of Archived Logs in backup set 1

Thrd	Seq	Low SCN	Low Time	Next SCN	Next Time
1	34	4723890	31-JUL-26	4766775	01-AUG-26

BS Key Type LV Size Device Type Elapsed Time Completion Time

2	Full		1.49G	DISK	00:00:02	01-AUG-26
BP Key: 2 Status: AVAILABLE Compressed: NO Tag: TAG20260801T080734						
Piece Name: C:\ORACLE_19C\DATABASE\024ULMU6_1_1						

List of Datafiles in backup set 2

File	LV	Type	Ckp SCN	Ckp Time	Abs Fuz	SCN	Sparse	Name
1		Full	4766792	01-AUG-26			NO	C:\ORACLE19C\ORADATA\ORCL\SYSTEM01.DBF
3		Full	4766792	01-AUG-26			NO	C:\ORACLE19C\ORADATA\ORCL\SYSAUX01.DBF
4		Full	4766792	01-AUG-26			NO	C:\ORACLE19C\ORADATA\ORCL\UNDOTBS01.DBF

c) Simulate Database Failure

Step 1: Find the USERS datafile

```
Name: Suresh Dahal/Roll: 23/Section: A
```

PL/SQL procedure successfully completed.

```
SQL> select file#, name from v$datafile;
```

FILE#	NAME
1	C:\ORACLE19C\ORADATA\ORCL\SYSTEM01.DBF
3	C:\ORACLE19C\ORADATA\ORCL\SYSAUX01.DBF
4	C:\ORACLE19C\ORADATA\ORCL\UNDOTBS01.DBF
5	C:\ORACLE19C\ORADATA\ORCL\PDBSEED\SYSTEM01.DBF
6	C:\ORACLE19C\ORADATA\ORCL\PDBSEED\SYSAUX01.DBF
7	C:\ORACLE19C\ORADATA\ORCL\USERS01.DBF
8	C:\ORACLE19C\ORADATA\ORCL\PDBSEED\UNDOTBS01.DBF
9	C:\ORACLE19C\ORADATA\ORCL\ORCLPDB\SYSTEM01.DBF
10	C:\ORACLE19C\ORADATA\ORCL\ORCLPDB\SYSAUX01.DBF
11	C:\ORACLE19C\ORADATA\ORCL\ORCLPDB\UNDOTBS01.DBF
12	C:\ORACLE19C\ORADATA\ORCL\ORCLPDB\USERS01.DBF
13	C:\ORACLE19C\ORADATA\ORCL\ORCLPDB\SURESH_DF1_23.DBF
14	C:\ORACLE19C\ORADATA\ORCL\ORCLPDB\SURESH_DF2_23.DBF

Step 2: Shutdown database

```
SQL> SHUTDOWN IMMEDIATE;
Database closed.
Database dismounted.
ORACLE instance shut down.
```

Step 3: Delete the USERS datafile

```
C:\Oracle19C\oradata\ORCL\orclpdb>del C:\ORACLE19C\ORADATA\ORCL\ORCLPDB\SU
RESH_DF2_23.DBF
```

Step 4: Switch to OCRLPDB and open database

```
SQL> alter database open
2 ;
alter database open
*
ERROR at line 1:
ORA-01157: cannot identify/lock data file 14 - see DBWR trace file
ORA-01110: data file 14: 'C:\ORACLE19C\ORADATA\ORCL\ORCLPDB\SURESH_DF2_23.
DBF'
```

d) Restore and Recover the Datafile

Step 1: Connect to RMAN

rman target /

```
C:\Oracle19C\oradata\ORCL\orclpdb>rman target /

Recovery Manager: Release 19.0.0.0.0 - Production on Sat Aug 1 10:30:28 20
26
Version 19.3.0.0.0

Copyright (c) 1982, 2019, Oracle and/or its affiliates. All rights reserv
ed.

connected to target database: ORCL (DBID=1764205357)

RMAN> |
```

Step 2: Restore the datafile

```
RMAN> restore datafile 14;

Starting restore at 01-AUG-26
allocated channel: ORA_DISK_1
channel ORA_DISK_1: SID=983 device type=DISK

channel ORA_DISK_1: starting datafile backup set restore
channel ORA_DISK_1: specifying datafile(s) to restore from backup set
channel ORA_DISK_1: restoring datafile 00014 to C:\ORACLE19C\ORADATA\ORCL\ORCLPDB\SURESH_DF2_23.DBF
channel ORA_DISK_1: reading from backup piece C:\ORACLE_19C\DATABASE\034ULMU9_1_1
channel ORA_DISK_1: piece handle=C:\ORACLE_19C\DATABASE\034ULMU9_1_1 tag=TAG20260801T080734
channel ORA_DISK_1: restored backup piece 1
channel ORA_DISK_1: restore complete, elapsed time: 00:00:01
Finished restore at 01-AUG-26
```

Step 3: Recover the datafile

```

RMAN> recover datafile 14;

Starting recover at 01-AUG-26
using channel ORA_DISK_1

starting media recovery
media recovery complete, elapsed time: 00:00:00

Finished recover at 01-AUG-26

```

Step 4: Verify

```

Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

SQL> select file#, status from v$datafile where file#=14;

  FILE# STATUS
  -----
      14 ONLINE

```

e) Full Database Recovery Simulation

Step 1: Shutdown database

```

SQL> exec my_info;
Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

SQL> shutdown immediate;
Pluggable Database closed.
SQL> |

```

Step 2: Delete the SPFILE, control files and datafiles

```

C:\>del "C:\Oracle_19C\database\SPFILEORCL.ORA"

C:\>del C:\ORACLE19C\ORADATA\ORCL\CONTROL01.CTL

C:\>del C:\ORACLE19C\ORADATA\ORCL\CONTROL02.CTL

C:\>del C:\ORACLE19C\ORADATA\ORCL\*.dbf

C:\>|

```

Step 3: Connect RMAN

```
C:\Users\VICTUS>rman target /

Recovery Manager: Release 19.0.0.0.0 - Production on Sat Aug 1 12:11:10 2026
Version 19.3.0.0.0

Copyright (c) 1982, 2019, Oracle and/or its affiliates. All rights reserved.

connected to target database: ORCL (not mounted)
```

Step 4: Restore SPFILE

```
RMAN> RESTORE SPFILE FROM AUTOBACKUP;

Starting restore at 01-AUG-26
using target database control file instead of recovery catalog
allocated channel: ORA_DISK_1
channel ORA_DISK_1: SID=539 device type=DISK

channel ORA_DISK_1: looking for AUTOBACKUP on day: 20260801
channel ORA_DISK_1: AUTOBACKUP found: c-1764205357-20260801-00
channel ORA_DISK_1: restoring spfile from AUTOBACKUP c-1764205357-20260801-00
channel ORA_DISK_1: SPFILE restore from AUTOBACKUP complete
Finished restore at 01-AUG-26
```

Step 5: Restore control file

```
RMAN> RESTORE CONTROLFILE FROM AUTOBACKUP;

Starting restore at 01-AUG-26
using channel ORA_DISK_1

channel ORA_DISK_1: looking for AUTOBACKUP on day: 20260801
channel ORA_DISK_1: AUTOBACKUP found: c-1764205357-20260801-00
channel ORA_DISK_1: restoring control file from AUTOBACKUP c-1764205357-20260801-00
channel ORA_DISK_1: control file restore from AUTOBACKUP complete
output file name=C:\ORACLE19C\ORADATA\ORCL\CONTROL01.CTL
output file name=C:\ORACLE19C\ORADATA\ORCL\CONTROL02.CTL
Finished restore at 01-AUG-26
```

Step 6: Restore database

```

RMAN> RESTORE DATABASE;

Starting restore at 01-AUG-26
using channel ORA_DISK_1

skipping datafile 5; already restored to file C:\ORACLE19C\ORADATA\ORCL\PDBSEED\SYSTEM01.DBF
skipping datafile 6; already restored to file C:\ORACLE19C\ORADATA\ORCL\PDBSEED\SYSAUX01.DBF
skipping datafile 8; already restored to file C:\ORACLE19C\ORADATA\ORCL\PDBSEED\UNDOTBS01.DBF
channel ORA_DISK_1: starting datafile backup set restore
channel ORA_DISK_1: specifying datafile(s) to restore from backup set
channel ORA_DISK_1: restoring datafile 00001 to C:\ORACLE19C\ORADATA\ORCL\SYSTEM01.DBF
channel ORA_DISK_1: restoring datafile 00003 to C:\ORACLE19C\ORADATA\ORCL\SYSAUX01.DBF
channel ORA_DISK_1: restoring datafile 00004 to C:\ORACLE19C\ORADATA\ORCL\UNDOTBS01.DBF
channel ORA_DISK_1: restoring datafile 00007 to C:\ORACLE19C\ORADATA\ORCL\USERS01.DBF
channel ORA_DISK_1: reading from backup piece C:\ORACLE_19C\DATABASE\024ULMU6_1_1
channel ORA_DISK_1: piece handle=C:\ORACLE_19C\DATABASE\024ULMU6_1_1 tag=TAG20260801T080734
channel ORA_DISK_1: restored backup piece 1
channel ORA_DISK_1: restore complete, elapsed time: 00:00:03
channel ORA_DISK_1: starting datafile backup set restore
channel ORA_DISK_1: specifying datafile(s) to restore from backup set
channel ORA_DISK_1: restoring datafile 00009 to C:\ORACLE19C\ORADATA\ORCL\ORCLPDB\SYSTEM01.DBF
channel ORA_DISK_1: restoring datafile 00010 to C:\ORACLE19C\ORADATA\ORCL\ORCLPDB\SYSAUX01.DBF
channel ORA_DISK_1: restoring datafile 00011 to C:\ORACLE19C\ORADATA\ORCL\ORCLPDB\UNDOTBS01.DBF
channel ORA_DISK_1: restoring datafile 00012 to C:\ORACLE19C\ORADATA\ORCL\ORCLPDB\USERS01.DBF
channel ORA_DISK_1: restoring datafile 00013 to C:\ORACLE19C\ORADATA\ORCL\ORCLPDB\SURESH_DF1_23.DBF
channel ORA_DISK_1: restoring datafile 00014 to C:\ORACLE19C\ORADATA\ORCL\ORCLPDB\SURESH_DF2_23.DBF
channel ORA_DISK_1: reading from backup piece C:\ORACLE_19C\DATABASE\034ULMU9_1_1
channel ORA_DISK_1: piece handle=C:\ORACLE_19C\DATABASE\034ULMU9_1_1 tag=TAG20260801T080734
channel ORA_DISK_1: restored backup piece 1
channel ORA_DISK_1: restore complete, elapsed time: 00:00:04
Finished restore at 01-AUG-26

```

Step 7: Recover database

```

RMAN> recover database;

Starting recover at 01-AUG-26
using channel ORA_DISK_1

starting media recovery

archived log for thread 1 with sequence 35 is already on disk as file C:\ORACLE19C\ORADATA\ORCL\REDO02.LOG
archived log for thread 1 with sequence 36 is already on disk as file C:\ORACLE19C\ORADATA\ORCL\REDO03.LOG
archived log file name=C:\ORACLE19C\ORADATA\ORCL\REDO02.LOG thread=1 sequence=35
archived log file name=C:\ORACLE19C\ORADATA\ORCL\REDO03.LOG thread=1 sequence=36
media recovery complete, elapsed time: 00:00:01
Finished recover at 01-AUG-26

```

Step 8: Open database

```

RMAN> ALTER DATABASE OPEN RESETLOGS;

Statement processed

```

f) Validation and Cleanup

Step 1: Validate database

```

RMAN> validate database;

Starting validate at 01-AUG-26
using channel ORA_DISK_1
channel ORA_DISK_1: starting validation of datafile
channel ORA_DISK_1: specifying datafile(s) for validation
input datafile file number=00001 name=C:\ORACLE19C\ORADATA\ORCL\SYSTEM01.DBF
input datafile file number=00003 name=C:\ORACLE19C\ORADATA\ORCL\SYSAUX01.DBF
input datafile file number=00004 name=C:\ORACLE19C\ORADATA\ORCL\UNDOTBS01.DBF
input datafile file number=00007 name=C:\ORACLE19C\ORADATA\ORCL\USERS01.DBF
channel ORA_DISK_1: validation complete, elapsed time: 00:00:01
List of Datafiles
=====
File Status Marked Corrupt Empty Blocks Blocks Examined High SCN
-----
1      OK      0              16718          116481          4788659
  File Name: C:\ORACLE19C\ORADATA\ORCL\SYSTEM01.DBF
  Block Type Blocks Failing Blocks Processed
  -----
  Data        0              81347
  Index       0              13330
  Other       0              5085

```

Step 2: Check corruption

```

RMAN> SELECT * FROM V$DATABASE_BLOCK_CORRUPTION;

no rows selected

```

Step 3: Verify data

```

Name: Suresh Dahal/Roll: 23/Section: A

PL/SQL procedure successfully completed.

SQL> SELECT username FROM dba_users;

USERNAME
-----
SYS
SYSTEM
XS$NULL
OJVMSYS
LBACSYS
OUTLN
SYS$UMF
DBSNMP
APPQOSSYS
DBSFUSER
GGSYS

```

12. LEARNING ABOUT STANDARD AUDITING IN ORACLE

THEORY

Standard Auditing in Oracle is a security feature used to monitor and record database activities performed by users. It helps database administrators track operations such as **SELECT**, **INSERT**, **UPDATE**, and **DELETE** on database objects. Audit records are stored in the database when the **AUDIT_TRAIL** initialization parameter is enabled. When configured as **DB**, audit records are stored in database tables, whereas **DB, EXTENDED** additionally stores the SQL statements executed. Standard auditing is commonly used to monitor user activities, improve database security, maintain accountability, and assist in compliance with organizational security policies.

STEPS and COMMANDS

(a) Preparation and Configuration for Standard Auditing

```
CONNECT / AS SYSDBA;
```

```
ALTER SESSION SET CONTAINER = ORCLPDB;
```

```
SHOW PARAMETER audit_trail;
```

(b) Enable Auditing

```
AUDIT SELECT, INSERT, UPDATE, DELETE  
ON HR.EMPLOYEES  
BY ACCESS;
```

(c) Generate Audit Records

```
CONNECT Hr/thaxaena@ORCLPDB;
```

```
SELECT first_name, email, salary  
FROM EMPLOYEES  
WHERE EMPLOYEE_ID = 100;  
  
UPDATE EMPLOYEES
```

```
SET SALARY = SALARY + 100  
WHERE EMPLOYEE_ID = 100;
```

```
COMMIT;
```

(d) Query and Analyze Audit Trail

```
CONNECT / AS SYSDBA;
```

```
ALTER SESSION SET CONTAINER = ORCLPDB;
```

```
SELECT username,  
       action_name,  
       obj_name,  
       timestamp,  
       sql_text  
FROM dba_audit_trail  
WHERE username='HR'  
AND obj_name='EMPLOYEES'  
ORDER BY timestamp DESC;
```

Column Description

- **USERNAME** – User who performed the action.
 - **ACTION_NAME** – Type of SQL operation performed.
 - **OBJ_NAME** – Name of the database object.
 - **TIMESTAMP** – Date and time of execution.
 - **SQL_TEXT** – SQL statement executed (available with DB, EXTENDED).
-

(e) Disable Auditing and Purge Audit Trail

```
NOAUDIT SELECT, INSERT, UPDATE, DELETE  
ON HR.EMPLOYEES;
```

```

BEGIN
  DBMS_AUDIT_MGMT.INIT_CLEANUP(
    AUDIT_TRAIL_TYPE => DBMS_AUDIT_MGMT.AUDIT_TRAIL_AUD_STD,
    DEFAULT_CLEANUP_INTERVAL => 24);
END;
/

BEGIN
  DBMS_AUDIT_MGMT.CLEAN_AUDIT_TRAIL(
    AUDIT_TRAIL_TYPE => DBMS_AUDIT_MGMT.AUDIT_TRAIL_AUD_STD,
    USE_LAST_ARCH_TIMESTAMP => FALSE);
END;
/

```

(f) Validation

```
CONNECT Hr/thaxaena@orclpdb;
```

```

SELECT first_name, email, salary
FROM EMPLOYEES
WHERE EMPLOYEE_ID = 100;
CONNECT / AS SYSDBA;

```

```
ALTER SESSION SET CONTAINER = ORCLPDB;
```

```

SELECT username,
       action_name,
       obj_name,
       timestamp
FROM dba_audit_trail
WHERE username='HR'

```

```
AND obj_name='EMPLOYEES'  
ORDER BY timestamp DESC;
```

OUTPUT

(a) Preparation and Configuration for Standard Auditing

```
Name: Suresh Dahal \ Roll: 23  
PL/SQL procedure successfully completed.  
  
SQL> CONNECT / AS SYSDBA;  
Connected.  
SQL>  
SQL> ALTER SESSION SET CONTAINER = ORCLPDB;  
  
Session altered.  
  
SQL>  
SQL> SHOW PARAMETER audit_trail;  
  
NAME                                TYPE                                VALUE  
-----                                -  
audit_trail                          string                             DB
```

(b) Enable Auditing

```
Name: Suresh Dahal \ Roll: 23  
PL/SQL procedure successfully completed.  
  
SQL> AUDIT SELECT, INSERT, UPDATE, DELETE  
2   ON HR.EMPLOYEES  
3   BY ACCESS;  
  
Audit succeeded.
```

(c) Generate Audit Records

```
SQL> connect hr/thaxaena@orclpdb  
Connected.
```

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> SELECT first_name, email, salary
2 FROM EMPLOYEES
3 WHERE EMPLOYEE_ID = 100;
```

FIRST_NAME	EMAIL	SALARY
Steven	SKING	24000

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> UPDATE EMPLOYEES
2 SET SALARY = SALARY + 100
3 WHERE EMPLOYEE_ID = 100;
```

1 row updated.

```
SQL> commit;
```

Commit complete.

(d) Query and Analyze Audit Trail

```
SQL> connect / as sysdba;
Connected.
SQL> ALTER SESSION SET CONTAINER = ORCLPDB;

Session altered.

SQL> set serveroutput on;
SQL> exec my_info;
Name: Suresh Dahal \ Roll: 23
```

Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

```
SQL> SELECT username,  
2         action_name,  
3         obj_name,  
4         timestamp,  
5         sql_text  
6 FROM dba_audit_trail  
7 WHERE username='HR'  
8 AND obj_name='EMPLOYEES'  
9 ORDER BY timestamp DESC;
```

USERNAME	ACTION_NAM	OBJ_NAME	TIMESTAMP	SQL_TEXT
HR	UPDATE	EMPLOYEES	01-AUG-26	
HR	UPDATE	EMPLOYEES	01-AUG-26	
HR	SELECT	EMPLOYEES	01-AUG-26	
HR	SELECT	EMPLOYEES	01-AUG-26	
HR	SELECT	EMPLOYEES	01-AUG-26	
HR	SELECT	EMPLOYEES	01-AUG-26	
HR	SELECT	EMPLOYEES	01-AUG-26	

(e) Disable Auditing and Purge Audit Trail

Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

```
SQL> NOAUDIT SELECT, INSERT, UPDATE, DELETE  
2 ON HR.EMPLOYEES;
```

Noaudit succeeded.

Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

```
SQL> BEGIN
  2   DBMS_AUDIT_MGMT.INIT_CLEANUP(
  3       AUDIT_TRAIL_TYPE => DBMS_AUDIT_MGMT.AUDIT_TRAIL_AUD_STD,
  4       DEFAULT_CLEANUP_INTERVAL => 24);
  5 END;
  6 /
```

Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

```
SQL> BEGIN
  2   DBMS_AUDIT_MGMT.CLEAN_AUDIT_TRAIL(
  3       AUDIT_TRAIL_TYPE => DBMS_AUDIT_MGMT.AUDIT_TRAIL_AUD_STD,
  4       USE_LAST_ARCH_TIMESTAMP => FALSE);
  5 END;
  6 /
```

PL/SQL procedure successfully completed.

(f) Validation

Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

```
SQL> connect hr/thaxaena@orclpdb;
Connected.
```

Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

```
SQL>
SQL> SELECT username,
  2         action_name,
  3         obj_name,
  4         timestamp
  5 FROM dba_audit_trail
  6 WHERE username='HR'
  7 AND obj_name='EMPLOYEES'
  8 ORDER BY timestamp DESC;
```

no rows selected

13. WORKING WITH UNIFIED AUDITING IN ORACLE

THEORY

Unified Auditing is Oracle's modern auditing framework that consolidates audit records from different sources into a single audit trail. It provides centralized monitoring of user activities, database operations, and security-related events. Audit policies are created to specify which actions should be monitored and can be enabled or disabled as needed. Audit records are stored in the UNIFIED_AUDIT_TRAIL view, allowing database administrators to analyze user activities, improve security, and meet compliance requirements.

STEPS and COMMANDS

(a) Preparation and Verification

Connect as SYS:

```
CONNECT / AS SYSDBA;
```

Switch to the PDB if using Oracle Multitenant:

```
ALTER SESSION SET CONTAINER = ORCLPDB;
```

Verify Unified Auditing:

```
SELECT parameter, value
```

```
FROM v$option
```

```
WHERE parameter = 'Unified Auditing';
```

If the value is FALSE, mention it in the report. Oracle 19c mixed mode still allows Unified Auditing.

(b) Create and Enable Audit Policy

Create the audit policy:

```
CREATE AUDIT POLICY hr_emp_policy
```

```
ACTIONS
```

```
SELECT ON HR.EMPLOYEES,
```

```
INSERT ON HR.EMPLOYEES,
```

```
UPDATE ON HR.EMPLOYEES,
```

```
DELETE ON HR.EMPLOYEES;
```

Enable the policy:

AUDIT POLICY hr_emp_policy;

(c) Generate Audit Records

Connect as HR:

CONNECT Hr/thaxaena;

Insert a record:

INSERT INTO employees

(employee_id, first_name, last_name, email, hire_date, job_id, salary)

VALUES

(999,

'Test',

'User',

'test@example.com',

SYSDATE,

'IT_PROG',

5000);

COMMIT;

Delete the record:

DELETE FROM employees

WHERE employee_id = 999;

COMMIT;

(d) Query and Analyze Unified Audit Trail

Connect as SYS:

CONNECT / AS SYSDBA;

Switch to the PDB:

```
ALTER SESSION SET CONTAINER = ORCLPDB;
```

Query the audit trail:

```
SELECT
    dbusername,
    action_name,
    object_schema,
    object_name,
    event_timestamp,
    sql_text
FROM unified_audit_trail
WHERE dbusername='HR'
AND object_name='EMPLOYEES'
ORDER BY event_timestamp DESC;
```

Explanation of Columns

- **DBUSERNAME** – Database user who performed the action.
 - **ACTION_NAME** – Type of SQL operation (INSERT, DELETE, UPDATE, SELECT).
 - **OBJECT_SCHEMA** – Owner of the audited object.
 - **OBJECT_NAME** – Name of the audited table.
 - **EVENT_TIMESTAMP** – Date and time when the event occurred.
 - **SQL_TEXT** – SQL statement that generated the audit record.
-

(e) Conditional Auditing

Create a conditional audit policy:

```
CREATE AUDIT POLICY hr_high_salary_pol
ACTIONS UPDATE ON HR.EMPLOYEES
WHEN 'NEW.SALARY > 10000'
EVALUATE PER STATEMENT;
```

Enable it:

AUDIT POLICY hr_high_salary_pol;

Connect as HR:

CONNECT Hr/thaxaena;

Execute a qualifying update:

UPDATE employees

SET salary = 12000

WHERE employee_id = 100;

COMMIT;

Verify the audit record:

CONNECT / AS SYSDBA;

ALTER SESSION SET CONTAINER = ORCLPDB;

SELECT

dbusername,

action_name,

event_timestamp,

sql_text

FROM unified_audit_trail

WHERE dbusername='HR'

ORDER BY event_timestamp DESC;

(f) Disable, Drop, and Cleanup

Disable both policies:

NOAUDIT POLICY hr_emp_policy;

NOAUDIT POLICY hr_high_salary_pol;

Drop them:

```
DROP AUDIT POLICY hr_emp_policy;
```

```
DROP AUDIT POLICY hr_high_salary_pol;
```

Purge the Unified Audit Trail:

```
BEGIN
```

```
    DBMS_AUDIT_MGMT.CLEAN_AUDIT_TRAIL(  
        AUDIT_TRAIL_TYPE => DBMS_AUDIT_MGMT.AUDIT_TRAIL_UNIFIED,  
        USE_LAST_ARCH_TIMESTAMP => FALSE);
```

```
END;
```

```
/
```

Validation:

Connect as HR:

```
CONNECT Hr/thaxaena;
```

```
SELECT *
```

```
FROM employees
```

```
WHERE employee_id = 100;
```

Connect as SYS:

```
CONNECT / AS SYSDBA;
```

```
ALTER SESSION SET CONTAINER = ORCLPDB;
```

Fetch Data

```
SELECT
```

```
    dbusername,
```

```
    action_name,
```

```
    object_name,
```

```
    event_timestamp
```

```
FROM unified_audit_trail
```

```
WHERE dbusername='HR'

ORDER BY event_timestamp DESC;
```

OUTPUT

(a) Preparation and Verification

```
SQL> show con_name;
```

```
CON_NAME
```

```
-----
```

```
ORCLPDB
```

```
Name: Suresh Dahal \ Roll: 23
```

```
PL/SQL procedure successfully completed.
```

```
SQL> SELECT parameter, value
       2 FROM v$option
       3 WHERE parameter = 'Unified Auditing';
```

```
PARAMETER
```

```
-----
```

```
VALUE
```

```
-----
```

```
Unified Auditing
```

```
FALSE
```

(b) Create and Enable Audit Policy

```
SQL> exec my_info;
```

```
Name: Suresh Dahal \ Roll: 23
```

```
PL/SQL procedure successfully completed.
```

```
SQL> CREATE AUDIT POLICY hr_emp_policy
```

```
2 ACTIONS
```

```
3 SELECT ON HR.EMPLOYEES,
```

```
4 INSERT ON HR.EMPLOYEES,
```

```
5 UPDATE ON HR.EMPLOYEES,
```

```
6 DELETE ON HR.EMPLOYEES;
```

```
Audit policy created.
```

Enable the policy:

```
Name: Suresh Dahal \ Roll: 23  
PL/SQL procedure successfully completed.  
SQL> AUDIT POLICY hr_emp_policy;  
Audit succeeded.
```

(c) Generate Audit Records

Connect as HR:

```
SQL> connect hr/thaxaena@orclpdb  
Connected.
```

Insert a record:

```
SQL> INSERT INTO employees  
2 (employee_id, first_name, last_name, email, hire_date, job_id, salary) VALUES (999, 'Test', 'User', 'test@example.com', SYSDATE, 'IT_PROG', 5000);  
1 row created.  
SQL> commit;  
Commit complete.
```

Delete the record:

```
Name: Suresh Dahal/ Roll: 23/ Section: A  
PL/SQL procedure successfully completed.  
SQL> DELETE FROM employees  
2 WHERE employee_id = 999;  
1 row deleted.  
SQL> commit;  
Commit complete.
```

(d) Query and Analyze Unified Audit Trail

```
SQL> connect / as sysdba;  
Connected.  
SQL> alter session set container = orclpdb;  
Session altered.
```

Query the audit trail:

```

Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

SQL> SELECT
  2     dbusername,
  3     action_name,
  4     object_schema,
  5     object_name,
  6     event_timestamp,
  7     sql_text
  8 FROM unified_audit_trail
  9 WHERE dbusername='HR'
 10 AND object_name='EMPLOYEES'
 11 ORDER BY event_timestamp DESC;

```

USER	ACTION	SCHEMA	OBJECT	TIMESTAMP	SQL QUERY
HR	DELETE	HR	EMPLOYEES	01-AUG-26 01.44.31.591000 PM	DELETE FROM employees WHERE employee_id = 999
HR	INSERT	HR	EMPLOYEES	01-AUG-26 01.43.41.331000 PM	INSERT INTO employees (employee_id, first_name, last_

name, email, hire_date, job

(e) Conditional Auditing

Create a conditional audit policy:

```

Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

SQL> CREATE AUDIT POLICY hr_hr_user_pol
  2 ACTIONS UPDATE ON HR.EMPLOYEES
  3 WHEN 'SYS_CONTEXT(''USERENV'', 'SESSION_USER') = ''HR''
  4 EVALUATE PER STATEMENT;

Audit policy created.

```

Enable it:

```

SQL> AUDIT POLICY hr_hr_user_pol;

Audit succeeded.

```

Connect as HR:

```
PS C:\Users\VICTUS> sqlplus hr/thaxaena@orclpdb;

SQL*Plus: Release 19.0.0.0.0 - Production on Sat Aug 1 13:54:08 2026
Version 19.3.0.0.0

Copyright (c) 1982, 2019, Oracle. All rights reserved.

Last Successful login time: Sat Aug 01 2026 13:39:23 +05:45

Connected to:
Oracle Database 19c Enterprise Edition Release 19.0.0.0.0 - Production
Version 19.3.0.0.0
```

Execute a qualifying update:

```
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> UPDATE employees
  2 SET salary = 12000
  3 WHERE employee_id = 100;

1 row updated.

SQL> commit
  2 ;

Commit complete.
```

Verify the audit record:

```
SQL> CONNECT / AS SYSDBA;
Connected.
SQL>
SQL> ALTER SESSION SET CONTAINER = ORCLPDB;

Session altered.
```

Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

```
SQL> SELECT
2     dbusername,
3     action_name,
4     event_timestamp,
5     sql_text
6 FROM unified_audit_trail
7 WHERE dbusername='HR'
8 ORDER BY event_timestamp DESC;
```

USER	ACTION	TIMESTAMP	SQL TEXT
HR	UPDATE	01-AUG-26 01.54.39.217000 PM	UPDATE employees SET salary = 12000 WHERE employee_id = 100
HR	LOGON	01-AUG-26 01.53.13.093000 PM	
HR	LOGON	01-AUG-26 01.53.02.855000 PM	
HR	LOGON	01-AUG-26 01.52.10.614000 PM	
HR	DELETE	01-AUG-26 01.44.31.591000 PM	DELETE FROM employees WHERE employee_id = 999

(f) Disable, Drop, and Cleanup

Disable both policies:

Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

```
SQL> NOAUDIT POLICY hr_emp_policy;
```

Noaudit succeeded.

```
SQL>
```

```
SQL> NOAUDIT POLICY hr_hr_user_pol;
```

Noaudit succeeded.

Drop them:

```
SQL> DROP AUDIT POLICY hr_emp_policy;
```

Audit Policy dropped.

```
SQL>
```

```
SQL> DROP AUDIT POLICY hr_hr_user_pol;
```

Audit Policy dropped.

Purge the Unified Audit Trail:

Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

```
SQL> BEGIN
  2     DBMS_AUDIT_MGMT.CLEAN_AUDIT_TRAIL(
  3         AUDIT_TRAIL_TYPE => DBMS_AUDIT_MGMT.AUDIT_TRAIL_UNIFIED,
  4         USE_LAST_ARCH_TIMESTAMP => FALSE);
  5 END;
  6 /
```

PL/SQL procedure successfully completed.

Validation:

Connect as HR:

```
PS C:\Users\VICTUS> sqlplus hr/thaxaena@orclpdb;
```

```
SQL*Plus: Release 19.0.0.0.0 - Production on Sat Aug 1 14:00:17 2026
Version 19.3.0.0.0
```

```
Copyright (c) 1982, 2019, Oracle. All rights reserved.
```

```
Last Successful login time: Sat Aug 01 2026 13:54:08 +05:45
```

```
Connected to:
```

```
Oracle Database 19c Enterprise Edition Release 19.0.0.0.0 - Production
Version 19.3.0.0.0
```

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> SELECT *
  2 FROM employees
  3 WHERE employee_id = 100;
```

	ID	FIRST NAME	LAST NAME	EMAIL	PHONE	HIRE DATE	JOB ID
D							
-							
0	100	Steven	King	SKING	515.123.4567	17-JUN-03	AD_PRES

Connect as SYS:

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> CONNECT / AS SYSDBA;

Connected.

SQL>

SQL> ALTER SESSION SET CONTAINER = ORCLPDB;

Session altered.

Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

SQL> SELECT

2 dbusername,

3 action_name,

4 object_name,

5 event_timestamp

6 FROM unified_audit_trail

7 WHERE dbusername='HR'

8 ORDER BY event_timestamp DESC;

no rows selected

14. EXPLORING FLASHBACK TECHNOLOGY IN ORACLE.

THEORY

Oracle Flashback Technology is a collection of features that helps recover data from accidental changes without restoring backups. It allows users to view, recover, or restore previous versions of data using information stored in the UNDO tablespace and archived redo logs. Flashback operations are faster than traditional recovery methods because they work directly on database objects instead of restoring the entire database.

Flashback Query retrieves data as it existed at a specific point in time by using the AS OF TIMESTAMP or AS OF SCN clause.

Flashback Version Query displays all versions of a row that existed during a specified time interval. It also shows the type of operation (INSERT, UPDATE, DELETE) and when each version became valid.

Flashback Table restores an entire table to a previous point in time without affecting other database objects. The table must have **ROW MOVEMENT** enabled before performing this operation.

Flashback Drop recovers accidentally dropped tables from the Oracle Recycle Bin. Instead of permanently deleting a table, Oracle stores it in the recycle bin until it is purged.

UNDO_RETENTION specifies how long Oracle attempts to preserve undo information. Flashback Query and Flashback Version Query rely on this undo data to retrieve previous versions of rows.

ARCHIVELOG mode archives redo log files, allowing complete recovery and supporting many Flashback features. The **Recycle Bin** stores dropped tables temporarily so they can be restored using Flashback Drop.

STEPS and COMMANDS

(a) Preparation and Configuration

Step 1: Connect as SYS

```
sqlplus / as sysdba
```

Step 2: Check UNDO_RETENTION

```
SHOW PARAMETER undo_retention;
```

Step 3: Check Recycle Bin

```
SHOW PARAMETER recyclebin;
```

Step 4: Set UNDO_RETENTION

```
ALTER SYSTEM SET undo_retention=3600;
```

Step 5: Verify the change

```
SHOW PARAMETER undo_retention;
```

Step 6: Verify ARCHIVELOG mode

```
ARCHIVE LOG LIST;
```

(b) Flashback Query**Step 1: Connect as HR**

```
CONNECT hr/thaxaena;
```

Step 2: Check current salary

```
SELECT employee_id,  
       first_name,  
       last_name,  
       salary  
FROM employees  
WHERE employee_id=103;  
Record the salary.
```

Step 3: Record current timestamp

```
SELECT SYSTIMESTAMP FROM dual;  
Record the timestamp.
```

Step 4: Increase salary

```
UPDATE employees  
SET salary = salary + 1000
```

WHERE employee_id=103;

COMMIT;

Step 5: Verify updated salary

```
SELECT employee_id,salary
FROM employees
WHERE employee_id=103;
```

Step 6: Flashback Query

```
SELECT employee_id,
       salary
FROM employees
AS OF TIMESTAMP
TO_TIMESTAMP(
'2026-08-01 07:31:44',
'YYYY-MM-DD HH24:MI:SS')
WHERE employee_id=103;
```

(c) Flashback Version Query

Step 1

Check salary.

```
SELECT employee_id,salary
FROM employees
WHERE employee_id=104;
```

Step 2

Increase salary by 500.

```
UPDATE employees
```

```
SET salary=salary+500  
WHERE employee_id=104;
```

```
COMMIT;
```

Step 3

```
SELECT SYSTIMESTAMP FROM dual;
```

Step 4

Increase salary again.

```
UPDATE employees  
SET salary=salary+600  
WHERE employee_id=104;
```

```
COMMIT;
```

Step 5

Display all versions.

```
SELECT versions_starttime,  
       versions_endtime,  
       versions_operation,  
       salary  
FROM employees  
VERSIONS BETWEEN  
TIMESTAMP  
SYSTIMESTAMP-INTERVAL '10' MINUTE  
AND SYSTIMESTAMP  
WHERE employee_id=104;
```

Explanation

- **VERSIONS_STARTTIME** – Time when that row version became active.
- **VERSIONS_ENDTIME** – Time when the version stopped being active.
- **VERSIONS_OPERATION** – Database operation (I, U, D).
- **SALARY** – Salary value for that version.

(d) Flashback Table

Step 1

Connect as SYS.

```
CONNECT / AS SYSDBA;
```

Step 2

Enable row movement.

```
ALTER TABLE hr.employees
ENABLE ROW MOVEMENT;
```

Step 3

Connect back as HR.

```
CONNECT hr/thaxaena;
```

Step 4

Delete employee 105.

```
DELETE
FROM employees
WHERE employee_id=105;
```

```
COMMIT;
```

Step 5

Record timestamp.

```
SELECT SYSTIMESTAMP FROM dual;
```

Step 6

Flashback table.

```
FLASHBACK TABLE employees  
TO TIMESTAMP  
TO _TIMESTAMP(  
'2026-08-01 07:43:08',  
'YYYY-MM-DD HH24:MI:SS');
```

Step 7

Verify.

```
SELECT first_name  
FROM employees  
WHERE employee_id=105;
```

(e) Flashback Drop

Step 1

Drop table.

```
DROP TABLE locations CASCADE CONSTRAINTS;
```

Step 2

Check recycle bin.

```
SHOW RECYCLEBIN;
```

Step 3

Restore table.

```
FLASHBACK TABLE locations  
TO BEFORE DROP;
```

Step 4

Verify.

```
SELECT *
```

```
FROM locations
```

```
WHERE location_id=1000;
```

(f) Validation and Cleanup

Restore employee 103 salary

```
UPDATE employees
```

```
SET salary=salary-1000
```

```
WHERE employee_id=103;
```

```
COMMIT;
```

Restore employee 104 salary

```
UPDATE employees
```

```
SET salary=salary-1100
```

```
WHERE employee_id=104;
```

```
COMMIT;
```

Delete employee 105 again

```
DELETE
```

```
FROM employees
```

```
WHERE employee_id=105;
```

```
COMMIT;
```

Demonstrating ORA-08180

```
SELECT first_name, last_name
```

```
FROM employees  
AS OF TIMESTAMP  
(SYSTIMESTAMP-INTERVAL '2' MONTH)  
WHERE employee_id=103;
```

Explanation:

A Flashback Query was executed using a timestamp two months in the past. Oracle returned ORA-08180: no snapshot was found based on specified time because the required undo information was no longer available. This indicates that Oracle could not reconstruct the table's state for the specified time.

OUTPUT

(a) Preparation and Configuration

Step 1: Connect as SYS

```
C:\Users\VICTUS>sqlplus / as sysdba  
  
SQL*Plus: Release 19.0.0.0.0 - Production on Sat Aug 1 19:25:26 2026  
Version 19.3.0.0.0  
  
Copyright (c) 1982, 2019, Oracle. All rights reserved.  
  
Connected to:  
Oracle Database 19c Enterprise Edition Release 19.0.0.0.0 - Production  
Version 19.3.0.0.0
```

Step 2: Check UNDO_RETENTION

```
Name: Suresh Dahal/Roll: 23/Section: A  
  
PL/SQL procedure successfully completed.  
  
SQL> SHOW PARAMETER undo_retention;  
  
NAME                                TYPE                                VALUE  
-----                                -  
undo_retention                      integer                             900
```

Step 3: Check Recycle Bin

Name: Suresh Dahal/Roll: 23/Section: A

PL/SQL procedure successfully completed.

SQL> SHOW PARAMETER recyclebin;

NAME	TYPE	VALUE
recyclebin	string	on

Step 4: Set UNDO_RETENTION (if less than 3600)

Name: Suresh Dahal/Roll: 23/Section: A

PL/SQL procedure successfully completed.

SQL> ALTER SYSTEM SET undo_retention=3600;

System altered.

Step 5: Verify the change

Name: Suresh Dahal/Roll: 23/Section: A

PL/SQL procedure successfully completed.

SQL> SHOW PARAMETER undo_retention;

NAME	TYPE	VALUE
undo_retention	integer	3600

Step 6: Verify ARCHIVELOG mode

Name: Suresh Dahal/Roll: 23/Section: A

PL/SQL procedure successfully completed.

SQL> ARCHIVE LOG LIST;

Database log mode	Archive Mode
Automatic archival	Enabled
Archive destination	C:\Oracle_19C\RDBMS
Oldest online log sequence	1
Next log sequence to archive	1
Current log sequence	1

(b) Flashback Query

Step 1: Connect as HR

```
SQL> connect hr/thaxaena@orclpdb;
Connected.
SQL> show user;
USER is "HR"
```

Step 2: Check current salary

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> SELECT employee_id,
2         first_name,
3         last_name,
4         salary
5 FROM employees
6 WHERE employee_id=103;
```

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
103	Alexander	Hunold	9000

Step 3: Record current timestamp

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> SELECT SYSTIMESTAMP FROM dual;
```

SYSTIMESTAMP

01-AUG-26 07.31.44.542000 PM +05:45

Step 4: Increase salary

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> UPDATE employees
  2  SET salary = salary + 1000
  3  WHERE employee_id=103;
```

1 row updated.

```
SQL> commit;
```

Commit complete.

Step 5: Verify updated salary

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> SELECT employee_id,salary
  2  FROM employees
  3  WHERE employee_id=103;
```

EMPLOYEE_ID	SALARY
103	10000

Step 6: Flashback Query

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> SELECT employee_id,
  2          salary
  3  FROM employees
  4  AS OF TIMESTAMP
  5  TO_TIMESTAMP(
  6  '2026-08-01 07:31:44',
  7  'YYYY-MM-DD HH24:MI:SS')
  8  WHERE employee_id=103;
```

EMPLOYEE_ID	SALARY
103	9000

(c) Flashback Version Query

Step 1

Check salary.

```
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> SELECT employee_id,salary
  2  FROM employees
  3  WHERE employee_id=104;

EMPLOYEE_ID      SALARY
-----
          104          6000
```

Step 2

Increase salary by 500.

```
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> UPDATE employees
  2  SET salary=salary+500
  3  WHERE employee_id=104;

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.
```

Step 3

```
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> SELECT SYSTIMESTAMP FROM dual;

SYSTIMESTAMP
-----
01-AUG-26 07.36.35.177000 PM +05:45
```

Step 4

Increase salary again.

```
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> UPDATE employees
  2 SET salary=salary+600
  3 WHERE employee_id=104;

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.
```

Step 5

Display all versions.

```
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> SELECT versions_starttime,
  2      versions_endtime,
  3      versions_operation,
  4      salary
  5 FROM employees
  6 VERSIONS BETWEEN
  7 TIMESTAMP
  8 SYSTIMESTAMP-INTERVAL '10' MINUTE
  9 AND SYSTIMESTAMP
 10 WHERE employee_id=104;
```

START TIME	END TIME	OP	SALARY
01-AUG-26 07.37.00 PM		U	7100
01-AUG-26 07.36.09 PM	01-AUG-26 07.37.00 PM	U	6500
	01-AUG-26 07.36.09 PM		6000

(d) Flashback Table

Step 1

Connect as sys

```
SQL> connect / as sysdba
Connected.
SQL> show user
USER is "SYS"
```

Step 2

Enable row movement.

```
SQL> ALTER TABLE hr.employees
 2  ENABLE ROW MOVEMENT;

Table altered.
```

Step 3

Connect back as HR.

```
SQL> connect hr/thaxaena@orclpdb;
Connected.
SQL> show user;
USER is "HR"
```

Step 4

Delete employee 105.

```
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> DELETE
 2  FROM employees
 3  WHERE employee_id=105;

1 row deleted.

SQL>
SQL> COMMIT;

Commit complete.
```

Step 5

Record timestamp.

```
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> SELECT SYSTIMESTAMP FROM dual;

SYSTIMESTAMP
-----
01-AUG-26 07.43.08.989000 PM +05:45
```

Step 6

Flashback table.

```
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> FLASHBACK TABLE employees
  2 TO TIMESTAMP
  3 TO_TIMESTAMP(
  4 '2026-08-01 07:43:08',
  5 'YYYY-MM-DD HH24:MI:SS');

Flashback complete.
```

Step 7

Verify.

```
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> select first_name from employees where employee_id = 105;

FIRST_NAME
-----
David
```

(e) Flashback Drop

Step 1

Drop table.

```

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> DROP TABLE locations CASCADE CONSTRAINTS;

Table dropped.

```

Step 2

Check recycle bin.

```

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> SHOW RECYCLEBIN;

```

ORIGINAL NAME	RECYCLEBIN NAME	OBJECT TYPE	DROP TIME
LOCATIONS	BIN\$w2EcpuQpQ1mevr8pPMxaww==\$0	TABLE	2026-08-01:19:49:48

Step 3

Restore table.

```

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> FLASHBACK TABLE locations
2 TO BEFORE DROP;

Flashback complete.

```

Step 4

Verify.

```

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> SELECT *
2 FROM locations
3 WHERE location_id=1000;

```

LOCATION_ID	STREET_ADDRESS	POSTAL_CODE	CITY	STATE_PROVINCE	CO
1000	1297 Via Cola di Rie	00989	Roma		IT

(f) Validation and Cleanup

Restore employee 103 salary

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> UPDATE employees
  2 SET salary=salary-1000
  3 WHERE employee_id=103;
```

1 row updated.

```
SQL>
SQL> COMMIT;
```

Commit complete.

Restore employee 104 salary

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> UPDATE employees
  2 SET salary=salary-1100
  3 WHERE employee_id=104;
```

1 row updated.

```
SQL>
SQL> COMMIT;
```

Commit complete.

Delete employee 105 again

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> DELETE
  2 FROM employees
  3 WHERE employee_id=105;
```

1 row deleted.

```
SQL>
SQL> COMMIT;
```

Commit complete.

Demonstrating ORA-08180

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> SELECT first_name, last_name
  2  FROM employees
  3  AS OF TIMESTAMP
  4  (SYSTIMESTAMP-INTERVAL '2' MONTH)
  5  WHERE employee_id=103;
FROM employees
  *
```

ERROR at line 2:

ORA-08180: no snapshot found based on specified time

15. ORACLE FLASHBACK VERSION QUERY AND TRANSACTION QUERY

THEORY

Oracle Flashback Technology provides mechanisms to view and recover historical data without restoring database backups. It uses information stored in the UNDO tablespace to reconstruct previous versions of rows and transactions. Flashback features are useful for recovering from accidental changes, auditing modifications, and analyzing data history.

Flashback Version Query displays every version of a row that existed within a specified time interval. It provides information such as when each version became active, the operation performed (Insert, Update, or Delete), the transaction ID (XID), and the values stored in each version. This feature is commonly used for auditing and tracking changes.

Flashback Transaction Query retrieves information about committed transactions from the FLASHBACK_TRANSACTION_QUERY view. It displays details such as the transaction ID, affected table, operation performed, commit timestamp, and automatically generated UNDO_SQL statements that can be executed to reverse the transaction.

The **UNDO_RETENTION** parameter determines how long Oracle attempts to retain undo information. Flashback features depend on the availability of this undo data. If the required undo information is no longer available, Oracle may return errors such as **ORA-08180** or **ORA-01555**.

STEPS and COMMANDS

(a) Preparation and Configuration

Step 1: Connect as SYSDBA

```
CONNECT / AS SYSDBA;
```

Step 2: Grant required privileges to HR

```
GRANT SELECT ANY TRANSACTION TO hr;
```

```
GRANT FLASHBACK ANY TABLE TO hr;
```

```
GRANT SELECT ON v_$transaction TO hr;
```

Step 3: Check UNDO retention

```
SHOW PARAMETER undo_retention;
```

Step 4: Increase UNDO retention (if less than 7200)

```
ALTER SYSTEM SET undo_retention=7200;
```

Step 5: Verify the parameter

```
SHOW PARAMETER undo_retention;
```

(b) Flashback Version Query Setup and Execution

Step 1: Connect as HR

```
CONNECT hr/thaxaena;
```

Step 2: View current salary

```
SELECT employee_id,  
       salary  
FROM employees  
WHERE employee_id=106;
```

Step 3: First update

```
UPDATE employees  
SET salary=salary+1000  
WHERE employee_id=106;
```

```
COMMIT;
```

Wait approximately **1 minute**.

Step 4: Second update

```
UPDATE employees  
SET salary=salary+500  
WHERE employee_id=106;
```

```
COMMIT;
```

Wait approximately **1 minute**.

Step 5: Delete the row

```
DELETE  
FROM employees
```

WHERE employee_id=106;

COMMIT;

Step 6: Display all versions

```
SELECT
    versions_starttime,
    versions_operation,
    versions_xid,
    salary
FROM employees
VERSIONS BETWEEN TIMESTAMP
SYSTIMESTAMP-INTERVAL '15' MINUTE
AND SYSTIMESTAMP
WHERE employee_id=106;
```

Explanation

- **VERSIONS_STARTTIME** shows when a row version became active.
 - **VERSIONS_OPERATION** indicates the operation performed:
 - **U** = Update
 - **D** = Delete
 - **I** = Insert
 - **VERSIONS_XID** is the transaction identifier associated with each version.
 - **SALARY** shows the salary value for each historical version.
-

(c) Analysis of Version Query Results

Step 1: Identify the transaction ID

From the Version Query output, note the **VERSIONS_XID** corresponding to the delete operation.

Step 2: Restore the deleted row

```
INSERT INTO employees
```

```
SELECT *  
FROM employees  
AS OF TIMESTAMP  
(  
    SYSTIMESTAMP-INTERVAL '5' MINUTE  
)  
WHERE employee_id=106;
```

COMMIT;

Step 3: Verify restoration

```
SELECT *  
FROM employees  
WHERE employee_id=106;
```

Discussion

Flashback Version Query enables users to view previous versions of data without maintaining manual audit tables or logs. It simplifies auditing by showing when changes occurred, the type of operation performed, and the transaction responsible for the modification.

(d) Flashback Transaction Query Setup and Execution

Step 1: Perform a transaction

```
UPDATE employees  
SET salary=salary+1000  
WHERE employee_id=107;
```

```
DELETE  
FROM employees  
WHERE employee_id=206;
```

Step 2: Obtain the transaction ID before committing

```
SELECT RAWTOHEX(xid) AS xid  
FROM v$transaction;
```

Record the XID displayed.

Step 3: Commit the transaction

COMMIT;

Step 4: Query Flashback Transaction Query

Replace **PASTE_XID_HERE** with the recorded XID.

```
SELECT
    operation,
    table_name,
    undo_sql
FROM flashback_transaction_query
WHERE xid=HEXTORAW(' 08001B00A8020000');
```

(e) Reversing Changes with Transaction Query

Execute the generated **UNDO_SQL** statements returned by the previous query.

Example:

```
INSERT INTO employees(...);
```

```
UPDATE employees
```

```
SET ...
```

```
WHERE ...;
```

Verify the changes

```
SELECT *
```

```
FROM employees
```

```
WHERE employee_id IN (107,108);
```

Explanation

UNDO_SQL contains SQL statements automatically generated by Oracle to reverse the original transaction. Executing these statements restores the data to its previous state. If undo information has expired, Oracle may be unable to reconstruct the required historical data and Flashback operations may fail.

(f) Validation and Advanced Scenario

Step 1: Attempt a Version Query beyond available undo history

```
SELECT *  
FROM employees  
AS OF TIMESTAMP  
(  
    SYSTIMESTAMP-INTERVAL '2' MONTH  
)  
WHERE employee_id=106;
```

Oracle may return either:

ORA-08180: no snapshot found based on specified time

or

ORA-01555: snapshot too old

depending on the availability of undo information.

Step 2: Perform a larger transaction

```
UPDATE employees  
SET salary=salary+500  
WHERE employee_id=109;
```

```
UPDATE departments  
SET manager_id=200  
WHERE department_id=20;
```

Step 3: Obtain the transaction ID

```
SELECT RAWTOHEX(xid) AS xid  
FROM v$transaction;
```

Step 4: Commit

```
COMMIT;
```

Step 5: Query transaction information

```
SELECT
```

operation,
table_name,
undo_sql

FROM flashback_transaction_query
WHERE xid=HEXTORAW(' 0A001000B7050000');

Step 6: Execute the generated UNDO_SQL statements

Execute the SQL statements returned by the query.

Step 7: Verify

SELECT employee_id,
salary
FROM employees
WHERE employee_id=109;

SELECT department_id,
manager_id
FROM departments
WHERE department_id=20;

OUTPUT

(a) Preparation and Configuration

Step 1: Connect as SYSDBA

```
C:\Users\VICTUS>sqlplus / as sysdba

SQL*Plus: Release 19.0.0.0.0 - Production on Sat Aug 1 20:18:08 2026
Version 19.3.0.0.0

Copyright (c) 1982, 2019, Oracle. All rights reserved.

Connected to:
Oracle Database 19c Enterprise Edition Release 19.0.0.0.0 - Production
Version 19.3.0.0.0

SQL> show user
USER is "SYS"
```

Step 2: Grant required privileges to HR

```

Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

SQL> GRANT SELECT ANY TRANSACTION TO hr;

Grant succeeded.

SQL>
SQL> GRANT FLASHBACK ANY TABLE TO hr;

Grant succeeded.

SQL>
SQL> GRANT SELECT ON v_$transaction TO hr;

Grant succeeded.

```

Step 3: Check UNDO retention

```

SQL> EXEC my_undo
Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

SQL> SHOW PARAMETER undo_retention;

```

NAME	TYPE	VALUE
undo_retention	integer	3600

Step 4: Increase UNDO retention (if less than 7200)

```

Name: Suresh Dahal \ Roll: 23

PL/SQL procedure successfully completed.

SQL> ALTER SYSTEM SET undo_retention=7200;

System altered.

```

Step 5: Verify the parameter

```

SQL> SHOW PARAMETER undo_retention;

```

NAME	TYPE	VALUE
undo_retention	integer	7200

(b) Flashback Version Query Setup and Execution

Step 1: Connect as HR

```
SQL> connect hr/thaxaena@orclpdb;  
Connected.
```

Step 2: View current salary

```
Name: Suresh Dahal/ Roll: 23/ Section: A  
  
PL/SQL procedure successfully completed.  
  
SQL> SELECT employee_id,  
2          salary  
3 FROM employees  
4 WHERE employee_id=106;  
  
EMPLOYEE_ID      SALARY  
-----  
106              4800
```

Step 3: First update

```
Name: Suresh Dahal/ Roll: 23/ Section: A  
  
PL/SQL procedure successfully completed.  
  
SQL> UPDATE employees  
2 SET salary=salary+1000  
3 WHERE employee_id=106;  
  
1 row updated.  
  
SQL>  
SQL> COMMIT;  
  
Commit complete.
```

Step 4: Second update

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> UPDATE employees
  2 SET salary=salary+500
  3 WHERE employee_id=106;
```

1 row updated.

SQL>

SQL> COMMIT;

Commit complete.

Step 5: Delete the row

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> DELETE
  2 FROM employees
  3 WHERE employee_id=106;
```

1 row deleted.

SQL>

SQL> COMMIT;

Commit complete.

Step 6: Display all versions

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> SELECT
  2     versions_starttime,
  3     versions_operation,
  4     versions_xid,
  5     salary
  6 FROM employees
  7 VERSIONS BETWEEN TIMESTAMP
  8 SYSTIMESTAMP-INTERVAL '15' MINUTE
  9 AND SYSTIMESTAMP
 10 WHERE employee_id=106;
```

VERSIONS_STARTTIME	V	VERSIONS_XID	SALARY
01-AUG-26 08.32.54 PM	D	0A000B00B7050000	6300
01-AUG-26 08.30.35 PM	U	09000B00B0020000	6300
01-AUG-26 08.25.34 PM	U	0A001500B6050000	5800
			4800

(c) Analysis of Version Query Results

Step 1: Identify the transaction ID

```
VERSIONS_XID
-----
0A000B00B7050000
09000B00B0020000
0A001500B6050000
```

Step 2: Restore the deleted row

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> INSERT INTO employees
  2 SELECT *
  3 FROM employees
  4 AS OF TIMESTAMP
  5 (
  6     SYSTIMESTAMP-INTERVAL '5' MINUTE
  7 )
  8 WHERE employee_id=106;
```

1 row created.

```
SQL>
SQL> COMMIT;
```

Commit complete.

Step 3: Verify restoration

```

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> select first_name, last_name from employees where employee_id = 106;

FIRST_NAME          LAST_NAME
-----
Valli               Pataballa

```

(d) Flashback Transaction Query Setup and Execution

Step 1: Perform a transaction

```

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> UPDATE employees
  2 SET salary=salary+1000
  3 WHERE employee_id=107;

1 row updated.

```

```

SQL> exec my_info;
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL>
SQL> DELETE
  2 FROM employees
  3 WHERE employee_id=206;

1 row deleted.

```

Step 2: Obtain the transaction ID before committing

```

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> SELECT RAWTOHEX(xid) AS xid
  2 FROM v$transaction;

XID
-----
02001E009B020000

```

Step 3: Commit the transaction

```
SQL> commit;

Commit complete.
```

Step 4: Query Flashback Transaction Query

```
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> SELECT
  2      operation,
  3      table_name,
  4      undo_sql
  5 FROM flashback_transaction_query
  6 WHERE xid=HEXTORAW('08001B00A8020000');

OPERATION  TABLE_NAME  UNDO_SQL
-----
UPDATE     EMPLOYEES  update "HR"."EMPLOYEES" set "SALARY" = '6200' wher
e ROWID = 'AAARywAAEAAAIwPAAR';

UPDATE     EMPLOYEES  update "HR"."EMPLOYEES" set "SALARY" = '9000' wher
e ROWID = 'AAARywAAEAAAIwPAAT';

UPDATE     EMPLOYEES  update "HR"."EMPLOYEES" set "SALARY" = '5200' wher
e ROWID = 'AAARywAAEAAAIwPAAR';
```

(e) Reversing Changes with Transaction Query

Execute the generated **UNDO_SQL** statements returned by the previous query.

```
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> update "HR"."EMPLOYEES" set "SALARY" = '6200' where ROWID = 'AAARywAAEAAAIwPAAR';

1 row updated.
```

```
SQL> update "HR"."EMPLOYEES" set "SALARY" = '9000' where ROWID = 'AAARywAAEAAAIwPAAT';

1 row updated.
```

Verify the changes

```
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> select first_name, last_name, salary from employees where employee_id in (107, 108);

FIRST_NAME      LAST_NAME      SALARY
-----
Diana           Lorentz        6200
Nancy           Greenberg      12008
```

(f) Validation and Advanced Scenario

Step 1: Attempt a Version Query beyond available undo history

```

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> SELECT *
  2 FROM employees
  3 AS OF TIMESTAMP
  4 (
  5     SYSTIMESTAMP-INTERVAL '2' MONTH
  6 )
  7 WHERE employee_id=106;
FROM employees
  *
ERROR at line 2:
ORA-08180: no snapshot found based on specified time

```

Step 2: Perform a larger transaction

```

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> UPDATE employees
  2 SET salary=salary+500
  3 WHERE employee_id=109;

1 row updated.

SQL>
SQL> UPDATE departments
  2 SET manager_id=200
  3 WHERE department_id=20;

1 row updated.

```

Step 3: Obtain the transaction ID

```

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> SELECT RAWTOHEX(xid) AS xid
  2 FROM v$transaction;

XID
-----
0A001000B7050000

```

Step 4: Commit

```
SQL> commit;

Commit complete.
```

Step 5: Query transaction information

```
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> SELECT
  2     operation,
  3     table_name,
  4     undo_sql
  5 FROM flashback_transaction_query
  6 WHERE xid=HEXTORAW('0A001000B7050000');

OPERATION  TABLE_NAME UNDO_SQL
-----
UPDATE     DEPARTMENT update "HR"."DEPARTMENTS" set "MANAGER_ID" = '201' where ROWID = 'AAARyAAEAAAIvtAAB';
S
UPDATE     EMPLOYEES   update "HR"."EMPLOYEES" set "SALARY" = '9000' where ROWID = 'AAARyAAEAAAIwPAAT';
UPDATE     EMPLOYEES   update "HR"."EMPLOYEES" set "SALARY" = '8900' where ROWID = 'AAARyAAEAAAIwPAAT';
UPDATE     EMPLOYEES   update "HR"."EMPLOYEES" set "SALARY" = '7200' where ROWID = 'AAARyAAEAAAIwPAAR';
BEGIN
```

Step 6: Execute the generated UNDO_SQL statements

```
SQL> update "HR"."EMPLOYEES" set "SALARY" = '9000' where ROWID = 'AAARyAAEAAAIwPAAT';

1 row updated.

SQL> update "HR"."EMPLOYEES" set "SALARY" = '8900' where ROWID = 'AAARyAAEAAAIwPAAT';

1 row updated.
```

Step 7: Verify

```
Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

SQL> SELECT employee_id,
  2         salary
  3 FROM employees
  4 WHERE employee_id=109;

EMPLOYEE_ID  SALARY
-----
109          8900

SQL>
SQL> SELECT department_id,
  2         manager_id
  3 FROM departments
  4 WHERE department_id=20;

DEPARTMENT_ID  MANAGER_ID
-----
20             200
```

16. AUTOMATIC WORKLOAD REPOSITORY (AWR) IN ORACLE 19C THEORY

Automatic Workload Repository (AWR) is a built-in Oracle performance monitoring and diagnostic tool that automatically collects, processes, and stores database performance statistics at regular intervals. It captures information such as CPU usage, wait events, SQL execution statistics, memory utilization, and system activity. AWR data helps database administrators analyze workload, identify performance bottlenecks, and optimize database performance.

AWR stores performance snapshots in the SYSAUX tablespace. These snapshots can be created automatically or manually. By comparing two snapshots, Oracle generates an AWR report that summarizes database performance during the selected time interval.

Important AWR parameters include:

- **STATISTICS_LEVEL** – Determines the level of statistics collection. It should be set to **TYPICAL** or **ALL** for AWR to function properly.
- **CONTROL_MANAGEMENT_PACK_ACCESS** – Enables Diagnostic and Tuning Packs. It should be set to **DIAGNOSTIC+TUNING**.
- **Snapshot Interval** – Specifies how frequently Oracle automatically creates AWR snapshots.

AWR reports contain useful sections such as Report Summary, Load Profile, Instance Efficiency Percentages, Top Timed Foreground Events, SQL Statistics, and Advisory Sections, which help identify inefficient SQL statements and system bottlenecks.

STEPS and COMMANDS

(a) Preparation and Configuration

Step 1: Connect to Oracle as SYSDBA

Open SQL*Plus and connect as SYSDBA.

```
CONNECT / AS SYSDBA;
```

Take a screenshot after successful login.

Step 2: Verify AWR parameters

Execute the following query to check whether AWR is properly configured.

```
SELECT name, value
```

```
FROM v$parameter
```

```
WHERE name IN
```

```
('statistics_level',  
'control_management_pack_access');
```

Verify that:

- STATISTICS_LEVEL = TYPICAL
- CONTROL_MANAGEMENT_PACK_ACCESS = DIAGNOSTIC+TUNING

Take a screenshot of the output.

Step 3: Modify parameters (only if required)

If any parameter is different, execute the appropriate command.

```
ALTER SYSTEM SET statistics_level='TYPICAL';
```

```
ALTER SYSTEM SET control_management_pack_access='DIAGNOSTIC+TUNING';
```

Run the query from Step 2 again to verify the changes.

Step 4: Check the default snapshot interval

Execute:

```
SELECT  
extract(day FROM snap_interval)*24*60+  
extract(hour FROM snap_interval)*60+  
extract(minute FROM snap_interval)  
AS snapshot_interval  
FROM dba_hist_wr_control;
```

The default snapshot interval is usually **60 minutes**.

Take a screenshot of the result.

(b) Create Baselines with Snapshots

Step 1: Create the first AWR snapshot

While connected as SYSDBA, create a manual snapshot.

```
EXEC DBMS_WORKLOAD_REPOSITORY.CREATE_SNAPSHOT;
```

Take a screenshot showing that the snapshot was created successfully.

Step 2: Connect as HR

Switch to the HR user.

```
CONNECT hr/hr;
```

Step 3: Generate workload

Execute the following SQL queries one after another.

```
SELECT e.employee_id,  
       e.first_name,  
       d.department_name  
FROM employees e  
JOIN departments d  
ON e.department_id=d.department_id;  
SELECT d.department_name,  
       COUNT(*)  
FROM employees e  
JOIN departments d  
ON e.department_id=d.department_id  
GROUP BY d.department_name;  
SELECT employee_id,  
       first_name,  
       salary  
FROM employees  
WHERE salary >  
(  
SELECT AVG(salary)  
FROM employees  
);
```

```
SELECT department_id,  
       AVG(salary)  
FROM employees  
GROUP BY department_id;  
  
SELECT *  
FROM employees  
ORDER BY salary DESC;
```

Next, execute the following update statement.

```
UPDATE employees  
SET salary = salary + 100  
WHERE employee_id = 101;
```

```
COMMIT;
```

Restore the original salary.

```
UPDATE employees  
SET salary = salary - 100  
WHERE employee_id = 101;
```

```
COMMIT;
```

Run the SELECT statements again 2–3 times to generate additional database activity.

Take screenshots while executing the workload.

Step 4: Create the second snapshot

Reconnect as SYSDBA.

```
CONNECT / AS SYSDBA;
```

Create another snapshot.

```
EXEC DBMS_WORKLOAD_REPOSITORY.CREATE_SNAPSHOT;
```

Take a screenshot.

Step 5: View available snapshots

Display all snapshots.

```
SELECT snap_id,  
       begin_interval_time,  
       end_interval_time  
FROM dba_hist_snapshot  
ORDER BY snap_id;
```

Note the two most recent SNAP_ID values because they will be used to generate the AWR report.

Take a screenshot.

(c) Generate AWR Report

Step 1: Run the AWR report script

While connected as SYSDBA, execute:

```
@?/rdbms/admin/awrrpt.sql
```

Oracle will ask several questions.

Step 2: Select report type

When prompted:

Specify the Report Type

Enter

html

Step 3: Specify the number of days

When prompted:

Specify the number of days of snapshots

Enter

1

Step 4: Enter Begin Snapshot ID

When prompted:

Enter value for begin_snap:

Enter the first SNAP_ID noted earlier.

Example:

150

Step 5: Enter End Snapshot ID

When prompted:

Enter value for end_snap:

Enter the second SNAP_ID.

Example:

151

Step 6: Specify the report name

When prompted:

Enter value for report_name:

Enter

awr_report.html

Oracle will generate the report and display a completion message similar to:

Report written to awr_report.html

Take a screenshot.

Step 7: Open the generated report

Open **awr_report.html** using any web browser.

(d) Analysis of AWR Report Sections

1. Instance Efficiency Percentages

- Buffer Hit % = **99.75%**, indicating efficient use of the buffer cache with very little disk I/O.
- Parse CPU to Parse Elapsed % = **78.13%**, indicating SQL parsing was reasonably efficient.
- Library Hit % = **88.47%**, showing most SQL statements reused existing execution plans.

2. Top Timed Foreground Events

- The highest wait event was **Failed Logon Delay (66.6%)**, followed by **DB CPU (30.5%)**.
- Disk I/O wait events were very low, indicating no major storage bottlenecks.

3. SQL Statistics

The SQL Statistics section shows that the top SQL statements were Oracle internal SQL and PL/SQL statements, such as `DBMS_WORKLOAD_REPOSITORY` and `DBMS_AUTO_REPORT_INTERNAL`, executed during snapshot creation and report generation. These SQL statements contributed approximately **49.2%** of the database activity, while PL/SQL contributed about **50.3%**.

4. Performance Analysis and Optimization

The AWR report shows a **Buffer Hit Ratio of 99.75%**, indicating efficient memory usage with very low disk I/O. However, the **Library Hit Ratio (88.47%)** and **Execute to Parse Ratio (61.37%)** suggest that SQL statements could be reused more efficiently. Performance can be improved by using **bind variables**, reducing unnecessary parsing, and optimizing frequently executed SQL statements.

OUTPUT

(a) Preparation and Configuration

Step 1: Connect to Oracle as SYSDBA

```
C:\Users\VICTUS>sqlplus / as sysdba

SQL*Plus: Release 19.0.0.0.0 - Production on Sat Aug 1 21:27:37 2026
Version 19.3.0.0.0

Copyright (c) 1982, 2019, Oracle. All rights reserved.

Connected to:
Oracle Database 19c Enterprise Edition Release 19.0.0.0.0 - Production
Version 19.3.0.0.0
```

Step 2: Verify AWR parameters

```
Name: Suresh Dahal/Roll: 23/Section: A

PL/SQL procedure successfully completed.

SQL> SELECT name, value
  2  FROM v$parameter
  3  WHERE name IN
  4  ('statistics_level',
  5  'control_management_pack_access');

NAME                                VALUE
-----
statistics_level                    TYPICAL
control_management_pack_access      DIAGNOSTIC
+TUNING
```

Step 3: Modify parameters

```
SQL> ALTER SYSTEM SET statistics_level='TYPICAL';

System altered.

SQL>
SQL> ALTER SYSTEM SET control_management_pack_access='DIAGNOSTIC+TUNING';

System altered.
```

Step 4: Check the default snapshot interval

Name: Suresh Dahal/Roll: 23/Section: A

PL/SQL procedure successfully completed.

```
SQL> SELECT
  2  extract(day FROM snap_interval)*24*60+
  3  extract(hour FROM snap_interval)*60+
  4  extract(minute FROM snap_interval)
  5  AS snapshot_interval
  6  FROM dba_hist_wr_control;
```

SNAPSHOT_INTERVAL

60

(b) Create Baselines with Snapshots

Step 1: Create the first AWR snapshot

Name: Suresh Dahal/Roll: 23/Section: A

PL/SQL procedure successfully completed.

```
SQL> EXEC DBMS_WORKLOAD_REPOSITORY.CREATE_SNAPSHOT;
```

PL/SQL procedure successfully completed.

Step 2: Connect as HR

```
SQL> connect hr/thaxaena@orclpdb;
Connected.
```

Step 3: Generate workload

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> SELECT e.employee_id,  
2         e.first_name,  
3         d.department_name  
4 FROM employees e  
5 JOIN departments d  
6 ON e.department_id=d.department_id;
```

EMPLOYEE_ID	FIRST_NAME	DEPARTMENT_NAME
200	Jennifer	Administration
201	Michael	Marketing
202	Pat	Marketing
114	Den	Purchasing
119	Karen	Purchasing
115	Alexander	Purchasing
116	Shelli	Purchasing
117	Sigal	Purchasing
118	Guy	Purchasing

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> SELECT d.department_name,  
2         COUNT(*)  
3 FROM employees e  
4 JOIN departments d  
5 ON e.department_id=d.department_id  
6 GROUP BY d.department_name;
```

DEPARTMENT_NAME	COUNT(*)
Sales	34
Marketing	2
Administration	1
Purchasing	6
Shipping	45
IT	4
Executive	3
Finance	6
Public Relations	1
Human Resources	1
Accounting	1

11 rows selected.

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> SELECT employee_id,  
2         first_name,  
3         salary  
4 FROM employees  
5 WHERE salary >  
6 (  
7 SELECT AVG(salary)  
8 FROM employees  
9 );
```

EMPLOYEE_ID	FIRST_NAME	SALARY
201	Michael	13000
203	Susan	6500
204	Hermann	10000
205	Shelley	12008
100	Steven	24000
101	Neena	17000
102	Lex	17000
103	Alexander	8000

Name: Suresh Dahal/ Roll: 23/ Section: A

PL/SQL procedure successfully completed.

```
SQL> SELECT department_id,  
2         AVG(salary)  
3 FROM employees  
4 GROUP BY department_id;
```

DEPARTMENT_ID	AVG(SALARY)
50	3475.55556
40	6500
110	12008
70	10000
90	19333.3333
30	4150
10	4400
	7000
20	9500
60	6225
100	8584.66667

Next, execute the following update statement.

```
Name: Suresh Dahal/ Roll: 23/ Section: A
```

```
PL/SQL procedure successfully completed.
```

```
SQL> UPDATE employees  
  2 SET salary = salary + 100  
  3 WHERE employee_id = 101;
```

```
1 row updated.
```

```
SQL>  
SQL> COMMIT;
```

```
Commit complete.
```

Restore the original salary.

```
Name: Suresh Dahal/ Roll: 23/ Section: A
```

```
PL/SQL procedure successfully completed.
```

```
SQL> UPDATE employees  
  2 SET salary = salary - 100  
  3 WHERE employee_id = 101;
```

```
1 row updated.
```

```
SQL>  
SQL> COMMIT;
```

```
Commit complete.
```

Step 4: Create the second snapshot

```
SQL> connect / as sysdba;  
Connected.  
SQL> EXEC DBMS_WORKLOAD_REPOSITORY.CREATE_SNAPSHOT;  
  
PL/SQL procedure successfully completed.
```

Step 5: View available snapshots

Display all snapshots.

Name: Suresh Dahal/Roll: 23/Section: A

PL/SQL procedure successfully completed.

```
SQL> SELECT snap_id,
2         begin_interval_time,
3         end_interval_time
4 FROM dba_hist_snapshot
5 ORDER BY snap_id;
```

SNAP_ID	BEGIN_INTERVAL_TIME	END_INTERVAL_TIME
375	24-JUL-26 06.46.04.429 AM	24-JUL-26 07.45.14.847 AM
376	24-JUL-26 07.45.14.847 AM	24-JUL-26 03.46.39.359 PM
377	24-JUL-26 03.46.39.359 PM	24-JUL-26 04.45.47.765 PM
378	24-JUL-26 04.45.47.765 PM	24-JUL-26 05.45.56.127 PM
379	24-JUL-26 05.45.56.127 PM	24-JUL-26 06.53.29.798 PM
380	24-JUL-26 06.53.29.798 PM	24-JUL-26 07.45.38.257 PM
381	24-JUL-26 07.45.38.257 PM	24-JUL-26 08.45.47.982 PM
382	24-JUL-26 08.45.47.982 PM	24-JUL-26 09.45.48.338 PM
383	24-JUL-26 09.45.48.338 PM	24-JUL-26 10.45.58.644 PM
384	24-JUL-26 10.45.58.644 PM	25-JUL-26 08.32.28.076 AM
385	25-JUL-26 08.32.28.076 AM	25-JUL-26 09.45.42.514 AM
386	25-JUL-26 09.45.42.514 AM	25-JUL-26 10.45.12.481 AM
387	25-JUL-26 10.45.12.481 AM	25-JUL-26 11.45.21.614 AM
388	25-JUL-26 11.45.21.614 AM	25-JUL-26 12.45.21.001 PM
389	25-JUL-26 12.45.21.001 PM	25-JUL-26 01.45.29.287 PM
390	25-JUL-26 01.45.29.287 PM	25-JUL-26 02.45.07.016 PM

477	01-AUG-26 04.45.38.636 PM	01-AUG-26 05.45.47.058 PM
478	01-AUG-26 05.45.47.058 PM	01-AUG-26 06.45.38.331 PM
479	01-AUG-26 06.45.38.331 PM	01-AUG-26 07.45.06.083 PM
480	01-AUG-26 07.45.06.083 PM	01-AUG-26 08.45.15.471 PM
481	01-AUG-26 08.45.15.471 PM	01-AUG-26 09.31.15.785 PM
482	01-AUG-26 09.31.15.785 PM	01-AUG-26 09.38.07.050 PM

(c) Generate AWR Report

Step 1: Run the AWR report script

```
SQL> @?/rdbms/admin/awrrpt.sql
```

Specify the Report Type

~~~~~

AWR reports can be generated in the following formats. Please enter the name of the format at the prompt. Default value is 'html'.

|               |                                        |
|---------------|----------------------------------------|
| 'html'        | HTML format (default)                  |
| 'text'        | Text format                            |
| 'active-html' | Includes Performance Hub active report |

#### Step 2: Select report type

```

SQL> @?/rdbms/admin/awrrpt.sql

Specify the Report Type
~~~~~
AWR reports can be generated in the following formats. Please enter the
name of the format at the prompt. Default value is 'html'.

'html' HTML format (default)
'text' Text format
'active-html' Includes Performance Hub active report

Enter value for report_type: html
old 1: select 'Type Specified: ',lower(nvl('&&report_type','html')) report_type from dual
new 1: select 'Type Specified: ',lower(nvl('html','html')) report_type from dual

Type Specified: html

```

### Step 3: Specify the number of days

```

Enter value for num_days: 1

Listing the last day's Completed Snapshots
Instance DB Name Snap Id Snap Started Snap Level

orcl ORCL 467 01 Aug 2026 00:45 1
 468 01 Aug 2026 07:07 1
 469 01 Aug 2026 07:45 1
 470 01 Aug 2026 09:52 1
 471 01 Aug 2026 10:45 1
 472 01 Aug 2026 12:36 1
 473 01 Aug 2026 13:45 1
 474 01 Aug 2026 14:45 1
 475 01 Aug 2026 15:45 1
 476 01 Aug 2026 16:45 1
 477 01 Aug 2026 17:45 1
 478 01 Aug 2026 18:45 1
 479 01 Aug 2026 19:45 1
 480 01 Aug 2026 20:45 1
 481 01 Aug 2026 21:31 1
 482 01 Aug 2026 21:38 1

```

### Step 4: Enter Begin Snapshot ID

```

Specify the Begin and End Snapshot Ids
~~~~~
Enter value for begin_snap: 481
Begin Snapshot Id specified: 481

```

### Step 5: Enter End Snapshot ID

```

Enter value for end_snap: 482
End Snapshot Id specified: 482

```

### Step 6: Specify the report name

```
Specify the Report Name
~~~~~
The default report file name is awrrpt_1_481_482.html. To use this name,
press <return> to continue, otherwise enter an alternative.

Enter value for report_name: awr_report.html

Using the report name awr_report.html
```

## Step 7: Open the generated report

### WORKLOAD REPOSITORY report for

| DB Name | DB Id      | Unique Name | Role    | Edition | Release    | RAC | CDB |
|---------|------------|-------------|---------|---------|------------|-----|-----|
| ORCL    | 1764205357 | orcl        | PRIMARY | EE      | 19.0.0.0.0 | NO  | YES |

| Instance | Inst Num | Startup Time    |
|----------|----------|-----------------|
| orcl     | 1        | 01-Aug-26 12:25 |

| Container DB Id | Container Name | Open Time       |
|-----------------|----------------|-----------------|
| 1764205357      | CDB\$ROOT      | 01-Aug-26 12:25 |
| 1793743613      | PDB\$SEED      | 01-Aug-26 12:25 |
| 3460620442      | ORCLPDB        | 01-Aug-26 12:53 |

| Host Name | Platform                     | CPUs | Cores | Sockets | Memory (GB) |
|-----------|------------------------------|------|-------|---------|-------------|
| ACONITIN  | Microsoft Windows x86 64-bit | 12   | 6     | 1       | 15.23       |

|             | Snap Id | Snap Time          | Sessions | Cursors/Session | Pluggable Databases Open |
|-------------|---------|--------------------|----------|-----------------|--------------------------|
| Begin Snap: | 481     | 01-Aug-26 21:31:15 | 61       | 1.3             | 3                        |
| End Snap:   | 482     | 01-Aug-26 21:38:07 | 61       | 1.3             | 3                        |
| Elapsed:    |         | 6.85 (mins)        |          |                 |                          |
| DB Time:    |         | 0.08 (mins)        |          |                 |                          |

## (d) Analysis of AWR Report Sections

### 1. Instance Efficiency Percentages

#### Instance Efficiency Percentages (Target 100%)

|                               |        |                   |        |
|-------------------------------|--------|-------------------|--------|
| Buffer Nowait %:              | 100.00 | Redo NoWait %:    | 100.00 |
| Buffer Hit %:                 | 99.75  | In-memory Sort %: | 100.00 |
| Library Hit %:                | 88.47  | Soft Parse %:     | 72.96  |
| Execute to Parse %:           | 61.37  | Latch Hit %:      | 99.87  |
| Parse CPU to Parse Elapsed %: | 78.13  | % Non-Parse CPU:  | 96.14  |
| Flash Cache Hit %:            | 0.00   |                   |        |

### 2. Top Timed Foreground Events

## Top 10 Foreground Events by Total Wait Time

| Event                        | Waits | Total Wait Time (sec) | Avg Wait  | % DB time | Wait Class |
|------------------------------|-------|-----------------------|-----------|-----------|------------|
| Failed Logon Delay           | 3     | 3                     | 1014.45ms | 66.6      | Other      |
| DB CPU                       |       | 1.4                   |           | 30.5      |            |
| Disk file operations I/O     | 33    | .1                    | 2.23ms    | 1.6       | User I/O   |
| control file sequential read | 244   | 0                     | 119.35us  | .6        | System I/O |
| db file sequential read      | 78    | 0                     | 287.86us  | .5        | User I/O   |
| log file sync                | 5     | 0                     | 944.00us  | .1        | Commit     |
| PGA memory operation         | 240   | 0                     | 7.87us    | .0        | Other      |
| PX Deq: Join ACK             | 2     | 0                     | 681.50us  | .0        | Other      |
| PX Deq: Slave Session Stats  | 4     | 0                     | 183.00us  | .0        | Other      |
| direct path write            | 10    | 0                     | 50.10us   | .0        | User I/O   |

## 3. SQL Statistics

### SQL ordered by Elapsed Time

- Resources reported for PL/SQL code includes the resources used by all SQL statements called by the code.
- % Total DB Time is the Elapsed Time of the SQL statement divided into the Total Database Time multiplied by 100
- %Total - Elapsed Time as a percentage of Total DB time
- %CPU - CPU Time as a percentage of Elapsed Time
- %IO - User I/O Time as a percentage of Elapsed Time
- Captured SQL account for 49.2% of Total DB Time (s): 5
- Captured PL/SQL account for 50.3% of Total DB Time (s): 5

| Elapsed Time (s) | Executions | Elapsed Time per Exec (s) | %Total | %CPU   | %IO   | SQL Id        | SQL Module  | PDB Name  | SQL Text                           |
|------------------|------------|---------------------------|--------|--------|-------|---------------|-------------|-----------|------------------------------------|
| 1.25             | 1          | 1.25                      | 27.28  | 91.40  | 5.60  | 491wcfyfd6wc1 | sqlplus.exe | CDB\$ROOT | BEGIN DBMS_WORKLOAD_REPOSITORY...  |
| 0.73             | 1          | 0.73                      | 16.02  | 94.91  | 0.84  | f1f8upax5cxsz |             | CDB\$ROOT | BEGIN sys.dbms_auto_report_int...  |
| 0.70             | 1          | 0.70                      | 15.22  | 93.34  | 0.88  | 0w26sk6t6qq98 |             | CDB\$ROOT | SELECT XMLTYPE(DBMS_REPORT.GET...  |
| 0.63             | 1          | 0.63                      | 13.73  | 93.15  | 0.98  | 9yv5dvw8k0awg |             | CDB\$ROOT | WITH MONITOR_DATA AS (SELECT I...  |
| 0.29             | 1          | 0.29                      | 6.26   | 100.04 | 0.59  | 6ajkhukk78nsr |             | CDB\$ROOT | begin prvt_hdm.auto_execute('...   |
| 0.13             | 3          | 0.04                      | 2.85   | 85.43  | 1.20  | dvu40a9avazf8 |             | CDB\$ROOT | select xmlagg( xmlelement("ope...  |
| 0.12             | 3          | 0.04                      | 2.53   | 87.15  | 2.33  | 3rdvj2msrkpn0 |             | CDB\$ROOT | select T.* from table(dbms_xpl...  |
| 0.06             | 1          | 0.06                      | 1.40   | 18.23  | 67.34 | dyycftbp20dc7 |             | CDB\$ROOT | insert into wrh\$_tablespace_st... |
| 0.05             | 644        | 0.00                      | 1.07   | 85.30  | 31.09 | f3ww8rgva3hrs |             | CDB\$ROOT | update /* KSMX.FLUSH COL */ sy...  |
| 0.05             | 1          | 0.05                      | 1.00   | 93.20  | 4.75  | fg91fmh6ux23c |             | CDB\$ROOT | insert into wrh\$_sga_target_ad... |

## 4. Performance Analysis and Optimization

### Instance Efficiency Percentages (Target 100%)

|                               |        |                   |        |
|-------------------------------|--------|-------------------|--------|
| Buffer Nowait %:              | 100.00 | Redo NoWait %:    | 100.00 |
| Buffer Hit %:                 | 99.75  | In-memory Sort %: | 100.00 |
| Library Hit %:                | 88.47  | Soft Parse %:     | 72.96  |
| Execute to Parse %:           | 61.37  | Latch Hit %:      | 99.87  |
| Parse CPU to Parse Elapsed %: | 78.13  | % Non-Parse CPU:  | 96.14  |
| Flash Cache Hit %:            | 0.00   |                   |        |